

SOME PHILOSOPHICAL ISSUES
concerning
THEORIES OF COMBINATORS

Dana Scott
Oxford University

Abstract. The paper presents in an informal way several conflicting viewpoints concerning concepts, theories, models, and applications of λ -calculus and the combinators.

Introduction. During the course of the Rome Symposium it became progressively more obvious that there is a very wide variety of motivations for studying λ -calculus and that the objectives of research need not be particularly well integrated. Perhaps that is as it should be, but perhaps too we should try to be a little clearer in making our standpoints understandable so that at least some measure of comparison is possible. In particular, we can see by listing the questionable points that many of the issues and features are, as some people say, orthogonal to one another. This means that wild mixtures are possible, and much confusion is probable, if we do not try to state from the outset how and why certain choices are being made.

There is no pretence here to having given a "definitive" philosophy of combinators, but the author's experience and observation covers a certain amount of ground from several fields. Thus by presenting a series of contrasts at not too great length, he hopes he will at least provoke people to ask themselves questions about the proper placing of their results and the direction of their work. Certainly one point seems definite: there is as yet no combinatory logic. This is much too absolute a term. The combinators appear in many rôles in many theories, but they have not given us an ultimate foundation either for logic or

for a theory of computation. They may never provide such foundations, but this is no criticism. They are interesting enough to merit study, and they deserve the favour of careful distinctions. Magic may have its place, but it is not here. There is more than sufficient evidence that they have a job to do - and if you do not fully understand how they do it, that is your fault not theirs.

In order to impose some slight organization of ideas, we shall divide the discussion into four parts: Concepts, Theories, Models, Applications. We intend here no hierarchy nor any indication of an historical progression. As a matter of fact people usually see the (rather formal) theories first, motivated by possible (and possibly unrealized) applications. It is only later (if ever) that they ask what concepts the theories were meant to formalize. The answer is not always so easy; sometimes models can help and often they fail - as they maybe hard to construct. It is also not easy to present the conceptual (pre-systematic) ideas apart from the theories and models in a way that is natural and makes the systematization seem inevitable. But that is usual in mathematics, and many people have good intuitions without being really able to explain them. This does not mean, however, that we should not try to give the necessary explanations. It is also true that many people have only vague ideas, and still they churn out endless numbers of "results"; but that is no reason to condone such activity. As the λ -calculus has now entered a "second period" of fairly wide-spread interest, we really ought to take some time for serious reflection.

§ 1. Concepts. The conceptual basis for the λ -calculus is the notion of function, and the combinators are certain very general functions that can be used to define new functions from old. So far so good, but we cannot stop here. Just because we employ functional notation and

discuss free and bound variables, it does not mean we know what we are doing. There are somany different kinds of functions, we are definitely obliged to say more. If we do not, then (just as with the pioneers of the subject) we should not be surprised to find that either our theories are too weak to be really useful or too strong to be consistent. This brings us to our first major contrast:

1.1 Typed vs. Untyped. The completely type-free logic is a will-O-the-wisp. This has been shown time and time again; yet the vision remains. And formal theories are quite as bad as drugs in keeping such illusions alive. The cold light of day reveals, however, perfectly solid ground. The notion of a type (as in type theory) is a sound concept. Functions do indeed have domains of definitions and ranges of values, and these ideas can be specified without having to say exactly which function is under consideration. Types are a way of making precise a portion of our ideas of separating functions into kinds; but this is only a start, since there are other properties besides domains and ranges that often need detailing.

A possible point of confusion concerns the relations between types. For example, one type may be more inclusive than another. Indeed, after becoming familiar with the usual finite types - often separated into a fairly rigid, noninclusive hierarchy - we may find it reasonable to introduce infinite types which include all the finite levels. This is done extensively in Zermelo-Fraenkel set theory, and for a certain kind of function concept this was done in the author's D_{∞} -models for λ -calculus. The point of confusion comes in because at these infinite levels we may think that we have achieved the goal of being type free. This is not the best way to explain the situation, however. Usually - with the aid of good notation - all we have done is to push the types into the background. They can always be brought out

again. When we write:

$$D_{\infty} = D_{\infty} \rightarrow D_{\infty}$$

we do not mean that D_{∞} is "type free". All we mean (after we say what " $\rightarrow$ " means) is that D_{∞} is a highly self-contained (or reflexive) domain and that certain operations do not take us outside the domain. The trick of course is to obtain a sufficiently high degree of "self-containedness". An absolute notion is impossible - unless one redefines what he expects of "absolute".

The moral then is to accept types, yet to realize how flexible the concept can be. Certain features of the type-free paradise are possible, but there are always limits. Speak not of the untyped but of the infinite types.

Having come to terms with natural limitations (which is not to say that one has a grasp of their scope), it is good to recall at this point a rather more specific contrast that has to do especially with functions:

1.2 Explicit vs. Recursive. Perhaps the terminology here is too specific. The point will become clear if we keep the finite types in mind. Even just with pure function types (all the type symbols generated from atomic ones by " $\rightarrow$ " alone), there are plenty of typed combinators. For example there are all the typed versions of I, S, and K. The question is: are there more? The answer is: no - if all we demand is closure under explicit definability - but it is yes if we also expect recursive definitions (as given, for example, by typed versions of the Y combinator). The problem is not one of what types to use, but rather one of what closure conditions to require.

It need hardly be stressed, but the same type symbols may have different interpretations in different theories. Constant use of the

"type-free" theories can make us insensitive to this obvious fact. The type-free theories also have the drawback that certain combinators - like the paradoxical combinator Y - give certain advantages "for free". Thus Y gives us recursion, which in a typed theory must be added as a new primitive. What else comes "for free" in the untyped theory? No one has ever provided a complete analyses; and maybe there is no final answer, since stronger and stronger theories could be possible.

In discussing closure conditions, we should not forget the conditional expression popularized by McCarthy. In giving recursion equations it is very convenient, and in some formulations it is needed a primitive as it is not definable in terms of the pure combinators. Another question about "truth-value" combinators will come up in 1.10 below.

1.3 Total vs. Partial. As soon as recursive definitions of any generality enter, the question of partial functions will arise. Often in the theory of types the underlying concept is that of total function. There is always a conflict which it is difficult to resolve. In the case of Kleene's recursion theory of higher-type functionals this conflict has often been remarked. In this admittedly infinitistic theory partial functions have a very secondary rôle which the (still unpublished) theory of Platek was meant to improve. Just how the traditional λ -calculus enters into these theories has only been partly explored (by Kleene and by Platek) and the work is not well known.

The type theories of de Bruijn and Martin-Löf (see the paper of de Vrijer in this symposium) also put the emphasis on total functions. The author's other paper in this symposium tries to suggest some connections with the "type-free" systems, but these proposals are rather tentative.

The author's lattice-theoretic models were conceived to accommodate partial functions from the start and the idea was heavily influenced by Platek's earlier work (there are also connections to Davis and Nerode). But the new point in the construction of the D_∞ model was that the spaces of finite type, the D_n , could be made cumulative - because we were using partially ordered spaces of partial functions. The trick of the partial ordering was that monotone well-defined (that is, total) functions give a model for a concept of partial function. There is a warning to take into account here to which we shall return in 1.4: the partially-ordered spaces which are so good for the fixed-point theorem may not model all aspects of how a function can be partial.

The issue is how to make clear the distinction between total and partial, and there is still very much left to say on the conceptual level.

1.4 Stable vs. Unstable. It was difficult to know what to call this contrast. It could be called "Curry vs. Kleene". It has to do with another aspect of partial functions. With Curry, application is always "meaningful" in the sense of always being allowed. With Kleene, the application $\{e\}(n)$ between numbers (here e is a Gödel number) is necessarily from time to time undefined. Curry's application is stable while Kleene's is unstable (this is meant to be descriptive not critical!)

The "modern" form of Kleene's theory is given with URS (see Barendregt's paper in this symposium). It is clear that Kleene carried over much of his experience with λ -calculus to these other structures, for example in his proof of his recursion theorem. Kleene also defined an analogous application $\{\alpha\}(\beta)$ for number-theoretic functions (see the Kleene-Vesley book). Troelstra has used the idea, but it does not

seem to have been studied "abstractly" like the URS.

The question is: what really is the connection between the two kinds of theories of combinators? Could the two theories have a common generalization? Or is one "better" than the other?

1.5 Strict vs. Inclusive. This contrast might also be called "Church vs. Curry". Church seemed to feel (as a consequence of the paradoxes?) that only expressions in normal form were meaningful. Curry on the other hand was willing to face all combinators. In his lecture at the Symposium Barendregt explained the situation very neatly in terms of the distinction between the λ -I-calculus and the λ -K-calculus. Church favoured the λ -I-calculus, and in this context, his feeling about normal forms can be made to look most reasonable. (The work Barendregt mentioned is to be published elsewhere.)

There is a definite conceptual point here: In the λ -I-calculus a function is meant to strictly depend on all its arguments. The question of dependence in logic has been very extensively discussed by Anderson and Belnap. The latter (in unpublished work) has tried to relate the λ -I-calculus to the logic (in the way Curry-Howard- et al. related combinators of the λ -K-calculus to intuitionistic logic) and he has discussed models. But the work seems as yet inconclusive. What is a strict function? Is it one which is undefined (or $\perp$) for undefined arguments? That seems like a necessary condition, but is that all we should say? There is a point to the λ -I-calculus, but more needs to be done to explain it.

1.6 Pure vs. Mixed. We might also call this " (η) vs. (ξ) ". In detailing one's concept of function, is one going to be moved to be so pure that absolutely everything is taken as being (uniquely) a func-

tion? That is what the (η) -rule means:

$$(\eta) \quad \lambda x. y(x) = y$$

Everything y is a function (namely: the function $\lambda x. y(x)$). Well, we know that this axiom is consistent (in pure λ -calculus), and it holds in the D_∞ -models (where, starting with the injection of D_0 into D_1 , everything is made to be a function). But is this axiom conceptually desirable? Might we not want to mix in some other sorts of elements among the functions?

It of course depends on what we want to do with the λ -calculus. We should, however, not confuse this point with that of 1.4. We can coherently make the application $y(x)$ stable even when y is not a function. In the author's graph model $(P\omega)$ we find instead of (η) :

$$(\eta^*) \quad y \subseteq \lambda x. y(x)$$

and this inclusion had an interesting connection with closure operators. There seems to be no trouble in "coercing" objects y to closely related functions $\lambda x. y(x)$, but why identify them? In an "unstable" theory we might even want $y(x)$ to be "meaningless" when y is not a function. Remember: being type-free does not mean having no type distinctions; rather it means having a large pot of accumulated elements with enough structure for a tasty stew. In cooking there are reasons to mix flavours, so why not in function theory? In any case, one should be conscious of what choices are possible.

1.7 Absolute vs. Relative. The combinators are great fun, but why is so much effort spent on pure λ -calculus? What reason is there to suppose that everything can be defined solely in terms of abstraction and application? As a matter of fact not everything can be so defined. Thus, should not more attention be given to relative definability, to definability in terms of given primitives? The automata people do it; the recursion-theory people do it; the logicians do it; and the

λ -people should too. If λ -calculus is ever to have any applications of any real importance, then it will be necessary. (Needless to say, there is no difficulty in talking about relative computability in the D_∞ and Pw models.) And, by the way, what ever became of the δ -rules that people used to discuss? Is there not some point to them?

1.8 Equational vs. Ordered. This point will come up again in Section 2 when we discuss theories, but there is a question of conceptual justification. Why spend all your thought on equations between combinators? The lattice-theoretic models have shown that there are very pleasant theories of partial orderings between λ -terms. The ideas connected with approximate normal forms, which were reported on in several papers in this symposium, have worked out very well and they essentially use inclusion and limits. In fact, how can we understand the fixed-point operator without limits? Of course, the study of partially ordered systems is more special than pure λ -calculus, but is it any less interesting? Might there not be other relations between terms that would also make sense? There is no reason that all useful properties can be expressed as equations - unless we add new operations (like $\cap$ or $\cup$ for lattices). More imagination about λ -calculus would be very welcome.

1.9 Compatible vs. Inconsistent. Even if a partial ordering is granted, there is a vexing question about forming unions of elements. Should $a \cup b$ (or, if you like: $a \sqcup b$) always make sense. Just as in the case of Curry vs. Kleene, where we might have reasons for having $x(y)$ undefined, we might feel that certain pairs of elements are inconsistent with each other. Do we want $a \cup b$ always to exist even when a and b somehow conflict?

In order to make the theory cleaner, Scott used lattices where

all sets of elements had joins - all elements were compatible with each other. This gives in particular his horrid "top" element $\top$. Now it is Scott vs. practically everyone else, since no one really likes $\top$. One reason to have a full lattice was the extension theorem for continuous functions. If D is a continuous lattice then any continuous function $f : X \rightarrow D$ can be extended to a continuous function $\bar{f} : Y \rightarrow D$ where Y is any superspace with $X \subseteq Y$ as a (topological) subspace. Indeed this property characterizes the continuous lattices, and it seemed very reasonable if we want rich models for partial functions. For example any partly defined continuous $f : A \rightarrow B$ where $A, B \subseteq D$, can be found as a restriction of an everywhere defined $\bar{f} : D \rightarrow D$ if D is a continuous lattice. If $D = D \rightarrow D$, we can see why it is nice to have one type of element $u \in D$ represent all manner of continuous functions on all subspaces of D . But all this argument shows is consistency. These lovely continuous lattices are possible, but what of other structures?

Several hours work did not give good definitions for the category of CPO's. We shall return to this question under a discussion of models. The point is from the conceptual side to say all you mean. It is not enough just to have the ideas of a partial ordering and of limits of chains because this is all that is needed for fixed-points. If one is to have a satisfactory theory of functions, functionals, objects both partial and total, then one has to face other questions about existence of elements. The $a \cup b$ question is one, and the extension question (which is closely related) is another having to do with completeness or fullness of the models. Much more investigation is needed in this area.

1.10 Deterministic vs. Nondeterministic. It is hard to know what to say on this score because current ideas are so vague. We certainly

have a feeling for the difference between the two kinds of computation; we can give examples of nondeterministically definable functions; but we cannot yet say that anyone has proposed very clear theoretical distinctions. Thus a union $a \cup b$ or a symmetric or in a boolean condition $p(x) \vee q(x)$ is nondeterministic (in that we have to try out both sides and cannot say in advance which will finish first). But this surely is not the whole story? There are other notions of search which will be needed (as in the theory of search computability of Moschovakis).

There is a great temptation to equate parallel computation with non-deterministic ideas, but much care is needed here. With parallelism there is sharing of data, and there is a need to resolve conflicts (inconsistencies) which does not seem to come up in simple nondeterminism. Despite a large number of papers in theoretical computer science on this problem area, the author remains very unsatisfied. When a really good answer is reached, however, he is sure that it will affect the λ -calculus, possibly with the introduction of new primitives that are as yet unrecognized.

1.11 Intensional vs. Extensional. In 1.10 we already spoke of computations. Up to this point we really only were questioning the property of computability (or definability). The latter is an extensional notion. It may turn out that the problems of parallelism and nondeterminism have only intensional solutions: the notions only make sense with regard to the way in which values are found. It may also be that problems of normal forms and Church-Rosser Theorems can only be discussed with particular computation rules in mind. If it turns out that way, fine! But there is so much confusion between use and mention, terms and denotations, form and content in writings on λ -calculus that it is often very difficult to know when you are standing on

solid ground. Thus as a final plea on conceptual clarity the author entreats people to make the necessary distinctions. It does not matter which side you are on as long as you know where you stand.

§ 2. Theories. The points to be brought up in the next two sections are much more technical, though some of them have already been touched upon. The main suggestion here is that people should be aware of the very wide variety of ways in which a theory can be formalized, since they are often far from equivalent.

2.1. Constants vs. Variables. There was a small movement at one time to ban the variable. Some philosophers still discuss it. The elimination of variables from logic is possible, but the price one has to pay in algebraic operators is very high indeed. Without casting our vote one way or the other, we can see there can be differences in point of view which make variables - at least free variables - natural. For example an extensionality rule like:

$$\frac{\vdash \tau(x) = \sigma(x)}{\vdash \tau = \sigma}$$

(where x is a "new" variable not free in σ or τ) does not mean the same as the ω -rule:

$$\frac{\vdash \tau(\xi) = \sigma(\xi) \text{ all terms } \xi}{\vdash \tau = \sigma}$$

(where the terms are all closed terms). For one thing the ω -rule is infinitistic and gives a highly nonelementary theory as was pointed out by Barendregt in his lecture. On the other hand the rule with the variable has a quite different import with regard to a model, since the hypothesis is meant to hold for all values of x in the model, not just for those that are definable by closed terms.

We are trying to say that variables are good or bad, but it does seem that principles can be expressed with their aid that cannot be

formulated without them. When the main emphasis has been on the pure combinators, variables have often been pushed to the sidelines and sometimes not given enough attention.

2.2. Equational vs. Conditional. It is the case that there are many curious and surprising equations that hold among the combinators, but these are of the status of completely general truths. In making theories and in proving theorems that are to have specialized applications, we often need rather more "local" truths. As a very minor example consider the obvious conditional:

$$u \circ u = u \vdash u \circ (u \circ u) = u ,$$

where $a \circ b = \lambda x. a(b(x))$. If the function u has its square equal to itself, then so with the cube (and every other positive power). All through algebra there are such conditionals to be found. There is no way to express this fact by a single equation. Many other examples can be found in the work of de Bakker and Milner. Thus interesting as the equations are, λ -calculus is more than just an equational theory.

2.3. Free vs. Quantified. The same concepts may be involved in many different theories. During the symposium Barendregt exhorted us to study combinatory algebras as algebras just as we do in mathematics with other structures. It is a good question whether anyone outside the "club" will want to listen to us, but that is a different problem. The point to be taken here is that the same structure can be studied in many ways. We know this in logic as regards first- and second-order arithmetic and in mathematics where, say, the reals also have a topology as well as an algebra. The point is a very obvious one, but sometimes it seems to be forgotten in studies of λ -calculus.

One reason for difficulties may be that on the level of theories (rather than models) there is generally no unique way to pass from the

lower-order theory to the higher one. In 2.2 we mentioned conditional equations, and it may well be the case that the set of all valid (or all desired) equations does not determine which conditional equations should be valid. (Of course, some are determined just from the conditional laws of equality). It is probably even worse when we go from free-variable theories to quantified theories.

As an example of a quantified, first-order theory, we have the axioms (α) , (β) , (ξ) mentioned by the author in his other paper for the symposium. It is in the extensionality axiom (ξ) that the quantifier enters. Off hand the author does not know whether there is a simple set of "combinatory" axioms that axiomatize the equations provable in this theory. He does not know an equivalent theory in terms of S and K (without λ). He has no idea how to axiomatize the quantifier-free part of this theory. All in all, it seems fair to say that the (first-order) quantified theory of combinators is an untouched area.

2.4. Algebraic vs. Inductive. Even if we do not care to pass to quantified theories, there are distinctions to make at the quantifier free level. Most formulations of λ -calculus are very algebraic. But there is no reason whatsoever to believe that the laws of reduction are the only laws for proving terms equal. Take the paradoxical combinator, for example:

$$Y = \lambda u. (\lambda x. u(x(x))) (\lambda x. u(x(x))) \quad .$$

It is defined in pure λ -language. In Scott's models this Y is the least fixed-point operator. From the work of Böhm it is known that there are many definitions of similar combinators that cannot be proved equal to Y by conversion alone. In the models all these combinators are equal. Many of the desired equations can, however, be proved by using Scott's Induction Rule. For instance, an interesting equation is:

$$Y(\lambda f \lambda x. g(x) (f(x))) = \lambda x. Y(g(x)) .$$

In Curry-style combinators we could write:

$$BYS = BY$$

The author has never attempted a proof, but it seems very likely that the methods of Böhm could be used to show that this cannot be proved by reduction. The proof by induction is very simple.

The conclusion is that if we really believe in the combinators, we may want to add an induction rule to our algebraic theories. This seems very reasonable if we want the combinators to help us with recursion theory. The reduction rules give us "for free" the recursion equations. But no one who has ever done any recursion theory would ever imagine that the equations were enough. Recursive definitions must always be joined with inductive proofs. It is just these natural proof methods that are usually missing from ordinary combinatory algebra. (The same could be said about the theory of URS.) It does not really seem right to call λ -calculus really a calculus if the induction rule (and some theory of limits) is missing. As far as it goes λ -algebra is fine, but it does not go far enough.

2.5. Finitary vs. Infinitary. Even though it is concerned with infinite processes, the induction rule is a finitary rule - at least in the sense that it gives a recursively enumerable set of theorems. We have already mentioned Barendregt's ω -rule which is essentially an infinitary rule. No doubt there could be infinitary versions of the induction rule. Going further, Fitch takes a highly infinitary view of combinatory "logic". This is discussed in the author's other paper as rather being a truth definition. (The distinction between syntax and semantics can sometimes blur when we pass to infinitary systems. The reason is that stronger rules make more notions absolute.) We

have, therefore, enough evidence to show that it is interesting to think about infinitary rules in the λ -calculus. It is another very underdeveloped area.

2.6. Classical vs. Non-classical. Here is an even less developed area. First there is three-valued logic that seems very appropriate to λ -calculus (see the author's other paper). Just how do theories work out in this logic? Next, remember that λ -calculus has quite reasonable interpretations restricted to computable functions. Thus we can certainly look at the theory from a constructive point of view. Constructivism is not the same as Intuitionism, but many of the aims are the same. Models of λ -calculus (or URS) can certainly be defined intuitionistically. Thus (especially in quantified theories) there is a question of how to give intuitionistic formalizations. Is there going to be a marked difference between classical and intuitionistic versions of λ -calculus? What about free-choice sequences and Brouwer's theorem on continuity of functions? There ought to be something to say on this score.

§ 3. Models. As it was unavoidable not to speak of models earlier, we shall be very brief here.

3.1. Terms vs. Values. If one's view of λ -calculus is very algebraic (even: equational), then after one has found a formally consistent theory, then it is not at all surprising to find that there are term models of the theory. But you get no more out of the term models than what you put into the theory. Term models do not by themselves provide any conceptual analysis nor do they give much indication of how to find applications of the theory. Model constructions that can be explained in advance of theory formalizations have a chance of doing both. It is not at all clear that "meaning as use" is an adequate philosophy. This is not to disparage formalism; but there is more to mathematics than

mere formalism. Even if one denies this, there is still the question of how to relate λ -calculus to other parts of mathematics (where functions are certainly considered important). Term models are precious little help for this purpose.

3.2. Constructive vs. Nonconstructive. The problem was really raised in 2.6: discuss the constructive aspects of λ -calculus models. On the other hand there may be interesting non-constructive models, too (see 3.5 below). Are term models constructive if the equality relation is not decidable? There may very well be a point that the URS version of combinators is more constructive (more intensional). In the P^ω -model, for example, there are many submodels. The least one is the class of recursively enumerable sets (constructive?) and every enumeration degree gives another model (see the authors Kiel paper). Which of these are constructive? More models can be found using the theory of retracts and the fixed-point construction. The constructive aspects have not been very much discussed (though work of Constable and Egli is oriented more in this direction).

3.3. CPO's vs. Lattices. This is a more technical point. We touched on it in 1.9 and 1.10. There is a strong feeling with many people that chain-complete partially ordered sets are enough and that continuous lattices are too much. The reason for taking continuous lattices was given in 1.9. There is a serious question here of how to judge the level of generality. Many CPO's can be completed to continuous lattices in a natural way (or at least embedded in them). Thus model constructions with CPO's may turn out just to be submodels of continuous lattices. So now who is to say which point of view is more general? Of course there may be an argument that (certain) CPO's are more economical models (having just the right kind of elements) and this

would be understandable. But the study has not gone forward far enough yet to be really conclusive. (As mentioned in 1.9 there is also a problem of defining further completeness properties of CPO's, another interesting question.)

3.4. Inverse Limits vs. Graph Models. Many people were sorry to see the author give up talking so much about the D_∞ construction. One reason was that inverse limits were rather tiresome to explain to people who had never thought about such algebraic constructions, and the necessary computations were quite long to give in complete detail. The advantage of the graph model P^ω was that its existence required no proof. The whole definition could be given in half a page. This does not mean that a person could understand the import of the construction in a flash, but at least he was not tired out even before he started.

Another advantage of the graph model is that it can be connected with ordinary recursion theory very easily. This is a little harder to see with inverse limits (whether for lattices or CPO's). Since, for the author at least, one of the reasons to develop λ -calculus is to have a simple and flexible way of presenting involved recursive definitions, it seems reasonable to relate things to the standard theories as quickly as possible so people can see how the new theory is a useful generalization.

The inverse limits are hardly lost, however. The graph model P^ω is universal for all separable continuous lattices: they are all retracts of P^ω . (All T_0 -spaces with a countably-based topology are also subspaces of P^ω as well.) Thus P^ω is a very rich model, and the author has shown (in the Kiel paper) that a calculus of retracts makes their definitions easy to find. Further, the retracts introduced by fixed-point constructions can often be related to inverse

limits. So the theory of these spaces is not really abandoned and they are even put in a wider context of recursively defined concepts.

3.5. Continuous vs. Monotone. Ordinary (finitary) recursions lead to continuous functions. The reason is, roughly, that the recursion is solved by finite iterations taken to a simple infinite limit. If we ever hope to be able to compute such functions, then we should keep to this continuous level. However, there are more infinitistic concepts: transfinite recursion for example.

In set theory, let H_ω be the set of hereditarily finite sets (all members of members of ... of members are finite). Let H_{ω_1} be the set of hereditarily countable sets. (H_{ω_1} is uncountable, while H_ω is countably infinite.) We can form graph models (almost exactly by Plotkin's original suggestion) by defining:

$$F(X) = \{y \mid \exists x \subseteq X. \langle x, y \rangle \in F\}$$

$$\lambda X. \tau = \{ \langle x, y \rangle \mid y \in \tau[x/X] \} .$$

Here the capital letters range over the power set (PH_ω or PH_{ω_1}) and the small letters over the small sets (H_ω or H_{ω_1}). The first model is our familiar model of continuous functions, while the second is the model of σ -continuous functions (continuous for unions of directed sets closed under countable sups). Such functions are monotone but only weakly continuous. PH_{ω_1} , however, is a very interesting λ -calculus model in which the Y -operator now does transfinite fixed points (no closure after ω in general).

We could go to higher cardinalities even to infinity in the strongest sense: let the X 's be arbitrary classes and the x 's just sets. We still have an interpretation of λ -calculus in, say, Gödel-Bernays set theory. (Note: no need for bound class variables.)

In the other direction we can find a host of submodels (even of

PH_{ω_1}). All of what is sketched here could be done over admissible sets. (Instead of arbitrary subsets, we should use just the Σ_1 -subsets to have a λ -calculus model.)

This seems to indicate there is a vast field for investigations of λ -calculus related to, say, descriptive set theory. There is certainly more to λ -calculus than meets the eye, even if we agree that too much has already met the eye in the published literature.

§ 4. Applications. It is difficult for the author not to write just about his personal interests. It is to be hoped that the other contributions to this symposium will provide a counterbalance. In concluding this paper, those applications that have especially concerned the author and have provided the motivation for him will be quickly reviewed.

The main interest that led to the discovery of the lattice-theoretic models was the need to find a basis for Strachey's theory of programming language semantics. The situation seems reasonably satisfactory at the moment and many people are working at giving language definitions in this style. A very great deal remains to be done to make the approach really practical, however, and by now the field is too large to survey here even superficially. The main effect of the λ -calculus models seems to have been - in very general terms - to give people more imagination about recursive definitions on abstract structures.

The relative success with programming language semantics suggests finding more connections with classical recursion theory. Though there have been many papers on abstract recursion theories of various sorts, no very unified view has come out. Possibly one lack is a good discussion of functionals, and here is a place that λ -calculus might

find an application. Another place to look for connections is a study of degrees other than the Turing degrees. The so-called enumeration degrees of sets of integers are very easily defined in the P_ω model and might merit more study. In another direction we could look for more connections with notions of computational complexity. Before starting on such projects, it would be well to consider the gains expected. It is unfortunate, but much of recursion theory leads a very isolated existence.

A very brief indication of how λ -calculus could be interpreted in set theory was mentioned in 3.5. This also might provide some applications. It is doubtful, however, that λ -calculus will lead to a particularly surprising alternative foundation for set theory as was once hoped.

In logic, λ -calculus is tied very closely to the theory of types. Infinite and transfinite types look interesting and this also brings us to category theory which is very close to type theory. Indeed the λ -calculus models give us new cartesian-closed categories, and if one only knew what conclusions to draw, one could find applications here. There is also proof theory which has been closely connected to λ -calculus. We would be grateful for a few new ideas at this point, however.